

Index

Symbols

\$ (dollar) sign, [Executing Node Code](#)
* as syntax, [ES Modules](#)
--production flag, [Using Packages](#)
--save-dev (-D) argument, [Using Packages](#)
. (relative path), [Resolving Modules](#)
./ (dot slash), [ES Modules](#)
.bat file extension, [The execFile Function](#)
.catch function, [Promises](#)
.cjs file extension, [ES Modules](#)
.cmd file extension, [The execFile Function](#)
.js file extension, [ES Modules](#)
.mjs file extension, [ES Modules](#)
.then function, [Promises](#)
== operator, [Assertions and Runners](#)
?? (nullish) operator, [Environment Variables](#)
[] (square brackets), [Node CLI](#)
\ (split into multiple lines), [Environment Variables](#)
^ (caret) character, [Semantic Versioning](#)
~ (tilde) character, [Semantic Versioning](#)

A

aborting, [Node CLI](#)
Active LTS (long-term support) status, [Executing Node Code](#)
add-on compiled files, [Loading Modules](#)
AI assistants, [Other Tools](#)
algorithms, [The Cluster Module](#)
ApacheBench (AB), [Primary and Worker Processes](#)

Apollo Server, [Frameworks](#)
application resources, minimizing, [Module Bundlers](#)
arguments keyword, [Scoping Modules](#) (see also options and arguments)
argv property, [Options and Arguments](#)
Arrange-Act-Assert pattern, [Assertions and Runners](#)
Array class, [REPL Mode](#)
assertion errors, [Custom Errors](#)
assertions, [Assertions and Runners](#)
async function, [The Non-Blocking Model](#)
async generators and iterators, [Async Generators and Iterators-Streams Object Mode](#)
async keyword, [Handler Functions](#)
async/await syntax, [Asynchronous Operations](#), [async/await-async/await](#)
asynchronous operations (see also event-driven model)
 callback pattern, [The Non-Blocking Model](#)
 event emitters, [Asynchrony-Asynchrony](#)
 event loop, [The Event Loop](#)
 import() function, [Asynchronous Operations](#)
 libuv library, [The Event Loop](#)
 Promise objects, [The Non-Blocking Model](#)
 sync versus async handling, [Sync Versus Async Handling-Sync Versus Async Handling](#)
authentication, [Handling State](#)
autocomplete, [REPL Mode](#)
automation
 automated testing, [Testing](#)
 loading modules automatically, [Options and Arguments](#)
 in Node, [Task Runners](#)
 task runners, [Task Runners](#)
availability, increasing, [Increasing Availability-Increasing Availability](#)
availableParallelism function, [Primary and Worker Processes](#)
await keyword, [The Non-Blocking Model](#), [async/await](#)

B

Babel, [JavaScript Transpilers](#)

backpressure, [Stream Events](#)
.bat file extension, [The execFile Function](#)
bidirectional streams, [Fundamentals of Streams](#)
brackets, square ([]), [Node CLI](#)
broadcasting messages, [Broadcasting Messages-Broadcasting Messages](#)
buffers, [Introducing Child Processes](#)

C

caching, [Caching Modules](#), [Handling State](#)
call stack, [The Event Loop](#)
callback hell, [Handler Functions](#)
callback pattern, [The Non-Blocking Model](#), [Handler Functions](#), [An Analogy for Promises](#)
caret (^) character, [Semantic Versioning](#)
.catch function, [Promises](#)
chaining, [Promises](#), [The pipeline Method](#)
child processes
 benefits of multiple processes, [Child Processes](#)
 creating, [Introducing Child Processes](#)
 execFile function, [The execFile Function](#)
 fork function, [The fork Function-The fork Function](#), [The Cluster Module](#)
 shell syntax and exec function, [Shell Syntax and the exec Function-Shell Syntax and the exec Function](#)
 spawn function, [The spawn Function-The spawn Function](#), [Shell Syntax and the exec Function](#)
 synchronous blocking functions, [The execFile Function](#)
.cjs file extension, [ES Modules](#)
clearInterval function, [Asynchronous Operations](#)
clearTimeout(timerId) function, [Asynchronous Operations](#)
cloning, [Strategies of Scalability](#), [Broadcasting Messages](#)
close event, [The spawn Function](#)
cluster.workers object, [Broadcasting Messages](#)
.cmd file extension, [The execFile Function](#)
code editors, [Other Tools](#)
code examples, obtaining and using, [Using Code Examples](#)
code quality, [Code Quality Tools](#), [Code Quality Tools-Other Tools](#)

code reviews, [Code Reviews](#)

commands (see also Node CLI)

aliasing, [Options and Arguments](#)

.break, [REPL Mode](#)

cd, [Executing Node Code](#)

.clear, [REPL Mode](#)

echo, [npm Run Scripts](#)

.editor, [REPL Mode](#)

exit, [REPL Mode](#), [npm Run Scripts](#)

find, [Introducing Child Processes](#)

grep, [Introducing Child Processes](#)

init, [Using Packages](#)

.load, [REPL Mode](#)

ls, [Using Packages](#)

mkdir, [Executing Node Code](#)

node, [Executing Node Code](#), [Node CLI](#)

node - C, [Node CLI](#)

node --env-file, [Environment Variables](#)

node --eval, [Options and Arguments](#)

node --import, [Options and Arguments](#)

node --print, [Options and Arguments](#)

node --require, [Options and Arguments](#)

node --test, [Options and Arguments](#)

node --v8-options | less, [Options and Arguments](#)

node --watch, [Options and Arguments](#)

npm, [Executing Node Code](#), [Node Packages](#), [The npm Command-](#)
[The npm Command](#)

npm -l, [The npm Command](#)

npm <command> -h, [The npm Command](#)

npm cache clean, [The npm Command](#)

npm create, [The npm Command](#)

npm help <term>, [The npm Command](#)

npm help npm, [The npm Command](#)

npm init, [Using Packages](#), [npm Run Scripts](#)

npm install, [Using Packages](#), [Using Packages](#), [The npm Command](#),
[The npm Command](#)

npm install <foo>, [The npm Command](#)

npm link, [The npm Command](#)
npm list <package>, [The npm Command](#)
npm list-unused-packages, [npm Run Scripts](#)
npm ls, [Updating and Removing Packages](#)
npm outdated, [Updating and Removing Packages](#)
npm pkg set type=module, [ES Modules](#)
npm prune, [Updating and Removing Packages](#)
npm publish, [The npm Command](#)
npm run, [npm Run Scripts](#)
npm run <foo>, [The npm Command](#)
npm run test, [npm Run Scripts](#)
npm search <search terms>, [The npm Command](#)
npm show, [The npm Command](#)
npm show <package>, [The npm Command](#)
npm test, [The npm Command](#), [npm Run Scripts](#)
npm uninstall, [Using Packages](#), [Updating and Removing Packages](#)
npm update, [Semantic Versioning](#), [Updating and Removing Packages](#)
npm-run-all, [Task Runners](#)
npx (node package execute), [The npx Command](#)
npx create-react-app, [The npx Command](#)
nvm install node, [Executing Node Code](#)
OS commands, [Introducing Child Processes](#)
ps, [Environment Variables](#)
pwd command, [The spawn Function](#)
reload, [Process Managers](#)
run npm eslint, [npm Run Scripts](#)
.save, [REPL Mode](#)
wc command, [The spawn Function](#)
comments and questions, [How to Contact Us](#)
CommonJS
versus ES modules, [Using Built-In Modules](#), [ES Modules](#)
exports argument, [Executing Modules](#)
implicit arguments in, [Scoping Modules](#)
module wrapping, [Scoping Modules](#)
potential drawbacks of, [Arguments Against Node](#)
using short absolute paths, [Resolving Modules](#)

compiled files, [Loading Modules](#)
configurable variables, [Executing Modules](#)
console object, [Executing Node Code](#)
console.log method, [Executing Node Code](#)
consuming streams, [Fundamentals of Streams](#), [The pipeline Method](#)
content management systems, [Frameworks](#)
context object, [Test Doubles](#)
continuous integration (CI), [Continuous Integration](#)
continuous integration and deployment services, [Task Runners](#)
coverage, [Continuous Integration](#)
CPU-bound tasks, [Arguments Against Node](#)
create-react-app package, [The npx Command](#)
createReadStream method, [Using Streams](#)
createServer function, [Using Built-In Modules](#)
crypto module, [Options and Arguments](#)
crypto stream, [Built-In Transform Streams](#)
Current release status, [Executing Node Code](#)
custom errors, [Custom Errors-Custom Errors](#)

D

Dahl, Ryan, [Introducing Node](#), [Node Built-In Modules](#)
data event, [Stream Events](#), [The spawn Function](#)
debugging
 Chrome browser DevTools, [Debugging in Node](#)
 fixing bugs, [Introducing Package Management](#)
 locating problems, [Debugging in Node](#)
 node inspect script.js, [Debugging in Node](#)
 NODE_DEBUG environment variable, [Environment Variables](#)
 performance profiler, [Debugging in Node](#)
 SIGUSR1 signal and, [Zero-Downtime Restarts](#)
 tracing flags, [Debugging in Node](#)
decomposing scalability, [Strategies of Scalability](#), [Broadcasting Messages](#), [Handling State](#)
deep equality check, [Assertions and Runners](#)
default export syntax, [ES Modules](#)
dependency injection design pattern, [Executing Modules](#)

dependency management methods, [Using Built-In Modules](#), [Using Packages](#), [Executing Modules](#), [Updating and Removing Packages](#)
dependency trees, [Introducing Package Management](#)
deployment services, [Task Runners](#)
describe method, [Assertions and Runners](#)
development frameworks, [Frameworks-Frameworks](#)
development-only dependencies (devDependencies), [Using Packages](#)
directories
 changing active, [Executing Node Code](#)
 creating new, [Executing Node Code](#)
disconnect event, [The spawn Function](#)
distributed caching, [Handling State](#)
dollar (\$) sign, [Executing Node Code](#)
dot slash (./), [ES Modules](#)
downloading, [Executing Node Code](#)
downtime, avoiding, [Zero-Downtime Restarts-Zero-Downtime Restarts](#)
drain event, [Stream Events](#)
dummy objects, [Test Doubles](#)
Duplex streams, [Fundamentals of Streams](#), [Duplex/Transform Streams](#)
dynamic import expressions, [Asynchronous Operations](#)
dynamically typed languages, [Arguments Against Node](#)

E

EACCES error code, [System Errors](#)
EADDRINUSE error code, [System Errors](#)
ECMAScript standard, [Using Built-In Modules](#), [JavaScript Transpilers](#)
ecommerce applications, [Frameworks](#)
ECONNRESET error code, [System Errors](#)
editors, [Other Tools](#)
emit function, [Event Emitters](#)
end event, [Stream Events](#)
end-to-end (e2e) test, [End-to-End Tests](#)
ENOENT error code, [System Errors](#)
ENOTFOUND error code, [System Errors](#)
entry points, [Module Bundlers](#)
env property, [Environment Variables](#)

environment variables, [Environment Variables-Environment Variables](#), [Resolving Modules](#)

equality, deep versus shallow, [Assertions and Runners](#)

error codes, [System Errors](#)

error-first callback style, [Handler Functions](#), [Layered Error Management](#)

errors

assertion errors, [Custom Errors](#)

custom errors, [Custom Errors-Custom Errors](#)

error events, [Errors](#), [Stream Events](#), [The spawn Function](#)

error forwarding method, [Layered Error Management](#)

EvalError, [TypeError](#)

input errors, [System Errors](#)

layered error management, [Layered Error Management-Layered Error Management](#)

logical type errors, [TypeScript](#)

operational errors, [System Errors](#)

preventing, [Preventive Measures](#)

RangeError, [RangeError](#)

ReferenceError, [ReferenceError](#)

standard errors, [Standard Errors-TypeError](#)

syntax type errors, [TypeScript](#)

SyntaxError, [SyntaxError](#)

system errors, [System Errors](#)

throwing and catching, [Throwing and Catching Errors-Throwing and Catching Errors](#)

timeout errors, [System Errors](#)

TypeError, [TypeError](#), [TypeScript](#)

types of, [Types of Errors-Custom Errors](#)

URIError, [TypeError](#)

user-specified errors, [Custom Errors](#)

ES modules

accessing filename or directory names of, [Scoping Modules](#)

await keyword, [async/await](#)

versus CommonJS, [Using Built-In Modules](#)

implicit scope execution, [Scoping Modules](#)

resolving but not executing, [Resolving Modules](#)

using, [ES Modules-ES Modules](#)

ESLint package

benefits of, [Code Quality Tools](#), [Code Quality Tools](#)

description of, [Node Packages](#)

development-only dependencies, [Using Packages](#)

using, [ESLint](#)

ETIMEDOUT error code, [System Errors](#)

EvalError, [TypeError](#)

event emitters

asynchrony, [Asynchrony-Asynchrony](#)

concept of, [Event Emitters](#)

creating, [Event Emitters](#)

error events, [Errors](#)

examples, [Examples](#)

main features of, [Event Emitters](#)

streams, [Fundamentals of Streams](#), [The spawn Function](#)

using, [Event Emitters](#)

event handlers, [Introducing Node](#)

event loop, [The Event Loop-The Event Loop](#)

event queue, [The Event Loop](#)

event-driven model

adopted by Node.js, [Introducing Node](#)

benefits of, [Asynchrony and Events](#), [Event Emitters](#)

defined, [Introducing Node](#)

event emitters, [Event Emitters-Examples](#), [Fundamentals of Streams](#)

event loop, [The Event Loop-The Event Loop](#)

handler functions, [Handler Functions-The Event Loop](#)

sync versus async handling, [Sync Versus Async Handling-Sync Versus Async Handling](#)

in V8 JavaScript engine, [Introducing Node](#)

events

flow of event loops, [Event Emitters](#)

implicit, [The Non-Blocking Model](#)

registering handlers for, [The spawn Function-The spawn Function](#)

slow operations and, [Introducing Node](#)

stream events, [Stream Events-Stream Events](#)

exec function, [Shell Syntax and the exec Function-Shell Syntax and the exec Function](#)
execFile function, [The execFile Function](#)
execute method, [Errors](#)
exit event, [The spawn Function](#), [The spawn Function](#)
explicit rest parameters, [Scoping Modules](#)
export object syntax, [ES Modules](#)
export statements, [ES Modules-ES Modules](#)
exports argument, [Executing Modules](#)
Express, [Frameworks](#)
extraneous label, [Updating and Removing Packages](#), [npm Run Scripts](#)

F

fake objects, [Test Doubles](#)
feature testing, [Functional Tests](#)
FIFO (first-in, first-out), [The Event Loop](#)
file-compression script, [Built-In Transform Streams](#)
files
 compressing, [Built-In Transform Streams](#)
 creating large, [Using Streams](#)
 locating, [ES Modules](#)
 types supported, [Loading Modules](#)
finish event, [Stream Events](#)
first-in, first-out (FIFO), [The Event Loop](#)
flowing mode (streams), [Paused and Flowing Modes](#)
for loops, [The Non-Blocking Model](#), [Sync Versus Async Handling](#), [The Event Loop](#)
fork function, [The fork Function-The fork Function](#), [The Cluster Module](#)
fs.readFile method, [Handler Functions](#), [Using Streams](#)
functional tests, [Functional Tests](#)
functions
 async function, [The Non-Blocking Model](#)
 availableParallelism function, [Primary and Worker Processes](#)
 .catch function, [Promises](#)
 clearTimeout(timerid) function, [Asynchronous Operations](#)
 createServer function, [Using Built-In Modules](#)

delaying execution of, [Asynchronous Operations](#)
emit function, [Event Emitters](#)
exec function, [Shell Syntax and the exec Function](#)-[Shell Syntax and the exec Function](#)
execFile function, [The execFile Function](#)
fork function, [The fork Function](#)-[The fork Function](#), [The Cluster Module](#)
ghRepos function, [Async Generators and Iterators](#)
handler functions, [Handler Functions](#)-[The Event Loop](#)
import() function, [Asynchronous Operations](#)
import.meta .resolve(), [Resolving Modules](#)
listener function, [Using Built-In Modules](#)
module.createRequire() function, [Loading Modules](#)
nested, [The Event Loop](#)
readFile function, [Examples](#)
readFileAsArray function, [Handler Functions](#), [Promises](#)
RequestListener function, [Using Built-In Modules](#)
require function, [Using Built-In Modules](#), [ES Modules](#), [Options and Arguments](#)
require.resolve() function, [Resolving Modules](#)
setInterval function, [Asynchronous Operations](#)
setTimeout function, [Asynchronous Operations](#), [The Non-Blocking Model](#), [Sync Versus Async Handling](#)
slowOperation function, [The Non-Blocking Model](#)
spawn function, [The spawn Function](#)-[The spawn Function](#), [Shell Syntax and the exec Function](#)
stacked, [The Event Loop](#)
.then function, [Promises](#)
timer functions, [The Non-Blocking Model](#)
unref function, [Shell Syntax and the exec Function](#)

G

generator packages, [The npx Command](#)
generators and iterators, [Async Generators and Iterators](#)-[Streams Object Mode](#)
ghRepos function, [Async Generators and Iterators](#)
GitHub Copilot, [Other Tools](#)

global scope objects, [Executing Node Code](#)
globalThis property, [Executing Node Code](#), [Scoping Modules](#)
GraphQL-based API servers, [Frameworks](#)
Grunt (task running tool), [Task Runners](#)
gulp (task running tool), [Task Runners](#)
gzipping, [Built-In Transform Streams](#)

H

handler functions

analogy for promises, [An Analogy for Promises-The Event Loop](#)
async/await syntax, [async/await-async/await](#)
asynchronous approach example, [Handler Functions](#)
child processes and, [The spawn Function](#)
error-first callback style, [Handler Functions](#), [Layered Error Management](#)
promises, [Promises-Promises](#)

harmony flags, [Options and Arguments](#)

Hello World example, [Using Built-In Modules](#)

help documentation

Node CLI, [Node CLI](#)
npm command, [The npm Command](#)
pm2 package, [Process Managers](#)
REPL mode, [REPL Mode](#)
tracing flags, [Debugging in Node](#)

Homebrew package manager, [Executing Node Code](#)

horizontal partitioning, [Strategies of Scalability](#)

horizontal scaling, [Strategies of Scalability](#)

HTTP/2 protocol, [Module Bundlers](#)

Husky, [Task Runners](#)

I

I/O operations, [The Non-Blocking Model](#)

immutable objects, [Immutable Objects](#)

implicit events, [The Non-Blocking Model](#)

import attributes, [Loading Modules](#)

import statements, [Using Built-In Modules](#), [ES Modules](#)-[ES Modules](#),
[Environment Variables](#)
import() function, [Asynchronous Operations](#)
import.meta .resolve() function, [Resolving Modules](#)
import/export statements, [Scoping Modules](#)
in-memory state, [Handling State](#)
index.js file, [ES Modules](#)
indirect dependencies, [Using Packages](#)
inference, [TypeScript](#)
input errors, [System Errors](#)
installation (see also npm package manager; packages)
 of Node.js, [Executing Node Code](#)
 of packages, [Using Packages](#)
integration tests, [Integration Tests](#)
interprocess communication (IPC), [The fork Function](#)
IP hash algorithm, [The Cluster Module](#), [Handling State](#)
isPrimary flag, [Primary and Worker Processes](#)
it method, [Assertions and Runners](#)
iterators and generators, [Async Generators and Iterators](#)-[Streams](#)
[Object Mode](#)

J

JavaScript language

 benefits of for Node.js, [The JavaScript Language](#)
 list of available globals, [REPL Mode](#)
 potential drawbacks of, [Arguments Against Node](#)
 transpilers for, [JavaScript Transpilers](#)
.js file extension, [ES Modules](#)

K

keywords

 arguments keyword, [Scoping Modules](#)
 async keyword, [Handler Functions](#)
 await keyword, [The Non-Blocking Model](#), [async/await](#), [async/await](#)
 with keyword, [Loading Modules](#)

L

last-in, first-out (LIFO), [The Event Loop](#)
least connections algorithm, [The Cluster Module](#)
libuv library, [The Event Loop](#)
Linux-based OS, [Executing Node Code](#)
listen method, [Using Built-In Modules](#)
listener function, [Using Built-In Modules](#)
listening event, [Examples](#), [Stream Events-Stream Events](#)
Live Server, [Task Runners](#)
load balancing, [The Cluster Module](#)
load testing, [Primary and Worker Processes](#)
lodash package, [Using Packages](#), [REPL Mode](#)
logical type errors, [TypeScript](#)

M

machine learning, [Other Tools](#)
macOS, [Executing Node Code](#)
Mailchimp Open Commerce, [Frameworks](#)
major number, [Semantic Versioning](#)
Math.random(), [Executing Node Code](#)
memory consumption, [Using Streams](#), [Stream Events](#)
memory management, [Options and Arguments](#)
message event, [The spawn Function](#)
messages, broadcasting, [Broadcasting Messages-Broadcasting Messages](#)
microservices, [Strategies of Scalability](#)
microtask queue, [The Event Loop](#)
middleware, [Frameworks](#)
minor number, [Semantic Versioning](#)
.mjs file extension, [ES Modules](#)
mock objects, [Test Doubles](#), [Test Doubles](#)
module bundlers, [Module Bundlers-Module Bundlers](#)
module.createRequire() function, [Loading Modules](#)
modules
 caching, [Caching Modules](#)

crypto module, [Options and Arguments](#)
definition of term, [Package Management](#)
descriptions of important, [Node Built-In Modules](#)
distinguishing external, [Using Built-In Modules](#)
ES modules, [Using Built-In Modules](#), [ES Modules-ES Modules](#),
[Resolving Modules](#), [Scoping Modules](#), [async/await](#)
executing, [Executing Modules-Executing Modules](#)
full list of, [Node Built-In Modules](#), [REPL Mode](#)
importing dynamically, [Asynchronous Operations](#)
loading, [Loading Modules](#)
loading automatically, [Options and Arguments](#)
managing, [Arguments Against Node](#)
node:assert module, [Assertions and Runners](#)
node:child_process module, [Examples](#), [Introducing Child Processes](#)
node:cluster module, [Examples](#), [The Cluster Module](#)
node:dgram module, [Examples](#)
node:fs module, [The Non-Blocking Model](#), [Options and Arguments](#),
[Using Streams](#)
node:http module, [Using Streams](#)
node:net module, [Examples](#)
node:os module, [Examples](#), [Primary and Worker Processes](#)
node:test module, [Testing](#), [Assertions and Runners](#)
node:timers module, [The Non-Blocking Model](#)
requiring, [Options and Arguments](#)
resolving, [Resolving Modules](#)
scoping, [Scoping Modules](#)
versus scripts, [Node Modules](#)
server.js module, [Using Built-In Modules](#), [ES Modules](#)
setting default type, [ES Modules](#)
types of, [Loading Modules](#)
using built-in, [Using Built-In Modules-Using Built-In Modules](#), [The Non-Blocking Model](#), [Node Built-In Modules-Node Built-In Modules](#),
[REPL Mode](#), [Resolving Modules](#), [Examples](#), [Fundamentals of Streams](#)
multiline mode, [REPL Mode](#)
multithreaded programming, [Introducing Node](#)

named exports, [ES Modules](#)

NestJS, [Frameworks](#)

network communication, [Fundamentals of Streams](#)

Node CLI (see also commands)

creating your own commands, [Options and Arguments](#)

environment variables, [Environment Variables-Environment Variables](#)

help pages, [Node CLI](#)

node command, [Node CLI](#)

options and arguments, [Options and Arguments-Options and Arguments](#)

REPL mode, [Executing Node Code](#), [REPL Mode-REPL Mode](#)

Node Version Manager (NVM), [Executing Node Code](#)

Node-API, [Loading Modules](#)

Node.js

approach to learning, [Why I Wrote This Book](#)

asynchronous operations, [Asynchronous Operations](#)

benefits of, [Preface](#), [Node Fundamentals](#)

executing Node code, [Executing Node Code-Executing Node Code](#), [Options and Arguments](#)

fundamentals of, [Node Fundamentals-Introducing Node](#)

non-blocking model, [The Non-Blocking Model-The Non-Blocking Model](#), [Asynchrony and Events](#), [The Event Loop](#)

packages, [Node Packages](#)

potential drawbacks of, [Arguments Against Node](#)

prerequisites to learning, [Who Should Read This Book](#)

use of JavaScript language, [The JavaScript Language](#)

using built-in modules, [Using Built-In Modules-Using Built-In Modules](#), [The Non-Blocking Model](#), [Node Built-In Modules-Node Built-In Modules](#), [REPL Mode](#), [Resolving Modules](#), [Examples](#), [Fundamentals of Streams](#)

using external packages, [Using Packages-Using Packages](#)

Node.js Toolbox website, [Introducing Package Management](#)

node: net module, [Examples](#)

node:assert module, [Assertions and Runners](#)

node:child_process module, [Examples](#), [Introducing Child Processes](#)
node:cluster module, [Examples](#), [The Cluster Module](#)
node:dgram module, [Examples](#)
node:fs module, [The Non-Blocking Model](#), [Options and Arguments](#),
[Using Streams](#)
node:fs/promises module, [The Non-Blocking Model](#)
node:http module, [Using Built-In Modules-Using Built-In Modules](#),
[Using Streams](#)
node:os module, [Examples](#), [Primary and Worker Processes](#)
node:test module, [Testing](#), [Assertions and Runners](#)
node:timers module, [The Non-Blocking Model](#)
NODE_DEBUG environment variable, [Environment Variables](#)
NODE_ENV environment variable, [Using Packages](#)
node_mksnapshot, [Node CLI](#)
node_modules folders, [Resolving Modules](#)
NODE_OPTIONS environment variable, [Environment Variables](#)
NODE_PATH environment variable, [Environment Variables](#), [Resolving Modules](#)
non-blocking model, [The Non-Blocking Model-The Non-Blocking Model](#), [Asynchrony and Events](#), [The Event Loop](#)
npm package manager, [Node Packages](#), [Introducing Package Management](#) (see also commands; packages)
nullish (??) operator, [Environment Variables](#)
NVM (Node Version Manager), [Executing Node Code](#)
nvm-windows, [Executing Node Code](#)

O

objectMode flag, [Streams Object Mode](#)
on method, [Event Emitters](#)
operational errors, [System Errors](#)
options and arguments
 error-first argument, [Handler Functions](#), [Layered Error Management](#)
 implicit arguments in CommonJS, [Scoping Modules](#)
 overview of, [Options and Arguments-Options and Arguments](#)
OS commands, [Introducing Child Processes](#)

package-lock.json file, [Updating and Removing Packages](#)
package.json files, [Using Packages](#), [Introducing Package Management](#),
[The npm Command](#)
packages
 creating and publishing, [Creating and Publishing Packages](#)-
 [Creating and Publishing Packages](#)
 definition of term, [Package Management](#)
 executing initializers, [The npm Command](#)
 finding and removing extraneous, [Updating and Removing Packages](#), [npm Run Scripts](#)
 list of, [Node Packages](#)
 managing, [Arguments Against Node](#)
 npm command, [The npm Command](#)-[The npm Command](#)
 npm loadtest package, [Primary and Worker Processes](#)
 npm package manager, [Node Packages](#), [Introducing Package Management](#)
 npm run scripts, [npm Run Scripts](#)-[npm Run Scripts](#)
 npx command, [The npx Command](#)
 package management, [Introducing Package Management](#)-
 [Introducing Package Management](#)
 package registries, [Node Packages](#)
 pm2 package, [Process Managers](#)
 semantic versioning, [Semantic Versioning](#)
 updating and removing, [Updating and Removing Packages](#)-
 [Updating and Removing Packages](#)
 using external, [Using Packages](#)-[Using Packages](#)
parallel processes, [Primary and Worker Processes](#)
passwords, generating, [Options and Arguments](#)
patch number, [Semantic Versioning](#)
paths, absolute versus relative, [Environment Variables](#), [Resolving Modules](#)
pause() method, [Paused and Flowing Modes](#)
paused mode (streams), [Paused and Flowing Modes](#)
peerDependencies, [Using Packages](#)
performance profiler, [Debugging in Node](#)

performance summaries, [Primary and Worker Processes](#), [Primary and Worker Processes](#), [Zero-Downtime Restarts](#)

piping

definition of term, [Introducing Streams](#)

paused and flowing modes, [Paused and Flowing Modes](#)

stream pipeline, [The pipeline Method](#)

to and from child processes, [Introducing Child Processes](#)

using pipe method, [Using Streams](#)

pm2 package, [Process Managers](#)

pressure, of streams, [Stream Events](#)

Prettier (code formatting tool), [Node Packages](#), [Code Quality Tools-Prettier](#)

primary process, [The Cluster Module-Primary and Worker Processes](#)

Print step, [Executing Node Code](#)

process IDs, [Primary and Worker Processes](#)

process managers, [Process Managers](#)

process object, [Environment Variables](#)

process.argv Array, [Options and Arguments](#)

processes, listing all running, [Environment Variables](#) (see also child processes)

--production flag, [Using Packages](#)

progress indicators, [Built-In Transform Streams](#)

Promise objects, [An Analogy for Promises-The Event Loop](#)

analogy for promises, [Promises-Promises](#)

versus callback functions, [The Non-Blocking Model](#)

forwarding errors, [Layered Error Management](#)

handling asynchronous operations, [Asynchronous Operations](#)

purpose of, [The Non-Blocking Model](#)

pull mode, [Paused and Flowing Modes](#)

push mode, [Paused and Flowing Modes](#)

pyramid of doom, [Handler Functions](#)

Q

questions and comments, [How to Contact Us](#)

R

random algorithm, [The Cluster Module](#)
random method, [Using Packages](#)
random strings, generating, [Options and Arguments](#)
RangeError, [RangeError](#)
read() method, [Paused and Flowing Modes](#)
Read-Eval-Print-Loop mode, [Executing Node Code](#), [REPL Mode-REPL Mode](#)
readable streams, [Introducing Streams](#), [Fundamentals of Streams](#), [Paused and Flowing Modes](#)
readFile function, [Examples](#)
readFileAsArray function, [Handler Functions](#), [Promises](#), [Event Emitters](#)
real-time applications, [Frameworks](#)
ReferenceError, [ReferenceError](#)
relative path (.), [Resolving Modules](#)
reload command, [Process Managers](#)
repetitive tasks, automating, [Task Runners](#)
REPL mode, [Executing Node Code](#), [REPL Mode-REPL Mode](#)
request event, [Examples](#)
RequestListener function, [Using Built-In Modules](#)
require function, [Using Built-In Modules](#), [ES Modules](#), [Options and Arguments](#)
require method, [Using Packages](#)
require.resolve() function, [Resolving Modules](#)
rest parameters, [Scoping Modules](#)
restarting, [Increasing Availability](#), [Zero-Downtime Restarts-Zero-Downtime Restarts](#)
results, printing, [Executing Node Code](#)
resume() method, [Paused and Flowing Modes](#)
round-robin algorithm, [The Cluster Module](#)
routing, [Frameworks](#)
routing algorithms, [The Cluster Module](#)
run scripts, [npm Run Scripts-npm Run Scripts](#)
runners, [Assertions and Runners-Assertions and Runners](#)

--save-dev (-D) argument, [Using Packages](#)

scaling

broadcasting messages, [Broadcasting Messages-Broadcasting Messages](#)

definition of term, [Scaling Node](#)

handling state, [Handling State](#)

increasing availability, [Increasing Availability-Increasing Availability](#)

node:cluster module, [The Cluster Module](#)

process managers, [Process Managers](#)

strategies of, [Strategies of Scalability](#)

zero-downtime restarts, [Zero-Downtime Restarts-Zero-Downtime Restarts](#)

scripts (see also commands; Node CLI)

defining names using pre or post prefixes, [npm Run Scripts](#), [Task Runners](#)

executing, [Node CLI](#), [Environment Variables](#)

file-compression script, [Built-In Transform Streams](#)

versus modules, [Node Modules](#)

npm run scripts, [npm Run Scripts-npm Run Scripts](#)

security issues, [Arguments Against Node](#)

semantic versioning (SemVer), [Semantic Versioning](#)

server.js module, [Using Built-In Modules](#), [ES Modules](#)

servers (see web servers)

sessions, managing, [Handling State](#)

setInterval function, [Asynchronous Operations](#)

setTimeout function, [Asynchronous Operations](#), [The Non-Blocking Model](#), [Sync Versus Async Handling](#)

shallow equality check, [Assertions and Runners](#)

sharding, [Strategies of Scalability](#)

shell commands, [npm Run Scripts](#)

shell syntax, [Shell Syntax and the exec Function-Shell Syntax and the exec Function](#)

SIGUSR2 signal, [Zero-Downtime Restarts](#)

slowOperation function, [The Non-Blocking Model](#)

snapshots, [Node CLI](#)

Socket.IO, [Frameworks](#)

spawn event, [The spawn Function](#)

spawn function, [The spawn Function](#)-[The spawn Function](#), [Shell](#)

[Syntax and the exec Function](#)

special characters, [Semantic Versioning](#)

split into multiple lines (), [Environment Variables](#)

split method, [Options and Arguments](#)

splitting, [Strategies of Scalability](#)

spy objects, [Test Doubles](#)

square brackets ([]), [Node CLI](#)

stacked functions, [The Event Loop](#)

standard errors, [Standard Errors](#)-[TypeError](#)

state management, [Handling State](#)

static typing tools, [Other Tools](#)

stdio streams, [Fundamentals of Streams](#)

stdout, [Executing Node Code](#)

sticky sessions, [Handling State](#)

Strapi, [Frameworks](#)

streams

analogies for, [Introducing Streams](#)

async generators and iterators, [Async Generators and Iterators](#)-
[Streams Object Mode](#)

built-in transform streams, [Built-In Transform Streams](#)

combining, [Introducing Streams](#), [Built-In Transform Streams](#)

data flow in, [Introducing Streams](#)

definition of term, [Introducing Streams](#)

fundamentals of, [Fundamentals of Streams](#)

implementing, [Implementing Streams](#)-[Duplex/Transform Streams](#)

major types of, [Introducing Streams](#)

objectMode flag, [Streams Object Mode](#)

paused and flowing modes, [Paused and Flowing Modes](#)

pipng, [Introducing Streams](#), [The pipeline Method](#)

stream events, [Stream Events](#)-[Stream Events](#), [The spawn Function](#)

supported by child processes, [Introducing Child Processes](#)

using, [Using Streams](#)-[Using Streams](#)

stub objects, [Test Doubles](#)

synchronous callbacks, [Handler Functions](#)
syntax checks, [Node CLI](#), [TypeScript](#)
SyntaxError, [SyntaxError](#)
system availability, increasing, [Increasing Availability-Increasing Availability](#)
system errors, [System Errors](#)

T

tab discoverability, [REPL Mode-REPL Mode](#)
target audience, [Who Should Read This Book](#)
task runners, [Task Runners](#)
TCP sockets, [Fundamentals of Streams](#)
templates, [The npx Command](#)
test-driven development (TDD), [Test-Driven Development](#)
testing
 assertions and runners, [Assertions and Runners-Assertions and Runners](#)
 automated, [Testing](#)
 benefits of regular, [Testing Node](#)
 candidates for, [Assertions and Runners](#)
 continuous integration, [Continuous Integration](#)
 example test with context, [Test Doubles](#)
 failing test example, [Assertions and Runners](#)
 frameworks for, [Other Tools](#)
 node --test command, [Options and Arguments](#)
 organizing and filtering tests, [Organizing and Filtering Tests-Organizing and Filtering Tests](#)
 passing test example, [Assertions and Runners](#)
 selecting tests, [End-to-End Tests](#)
 test doubles, [Test Doubles-Test Doubles](#)
 test-driven development, [Test-Driven Development](#)
 types of, [Types of Tests-End-to-End Tests](#)
 with limited resources and connectivity, [Module Bundlers](#)
 writing tests, [Testing](#)
.then function, [Promises](#)
threads, [Introducing Node](#), [Sync Versus Async Handling](#)
throttling mode, [Module Bundlers](#)

throw error call, [Throwing and Catching Errors](#), [Layered Error Management](#)

tilde (~) character, [Semantic Versioning](#)

timeout errors, [System Errors](#)

timer functions, [Asynchronous Operations](#), [The Non-Blocking Model](#)

tools

- automation in Node, [Task Runners](#)

- development frameworks, [Frameworks-Frameworks](#)

- JavaScript transpilers, [JavaScript Transpilers](#)

- module bundlers, [Module Bundlers-Module Bundlers](#)

- quality code tools, [Code Quality Tools-Other Tools](#)

- task runners, [Task Runners](#)

tracing flags, [Options and Arguments](#), [Debugging in Node](#)

transform streams, [Fundamentals of Streams](#), [Duplex/Transform Streams](#), [Built-In Transform Streams](#)

transitive dependency, [Introducing Package Management](#)

transpilers, [JavaScript Transpilers](#)

try/catch statement, [Promises](#), [Throwing and Catching Errors](#)

type conversions, [Assertions and Runners](#)

type import attribute, [Loading Modules](#)

type inference, [TypeScript](#)

type keys, [ES Modules](#)

TypeError, [TypeError](#), [TypeScript](#)

types

- built-in definitions for, [TypeScript](#)

- defining, [TypeScript](#)

- exporting/importing, [TypeScript](#)

TypeScript, [Node Packages](#), [Code Quality Tools](#), [JavaScript Transpilers-TypeScript](#)

U

unit tests, [Unit Tests](#), [Test Doubles](#)

unref function, [Shell Syntax and the exec Function](#)

updates

- JavaScript transpilers, [JavaScript Transpilers](#)

- npm update command, [Semantic Versioning](#)

for packages, [Updating and Removing Packages-Updating and Removing Packages](#)

trusting npm packages, [Introducing Package Management](#)

zero-downtime restarts, [Zero-Downtime Restarts](#)

URIError, [TypeError](#)

URL hash algorithm, [The Cluster Module](#)

user authentication, [Handling State](#)

user conditions, [Node CLI](#)

user-specified errors, [Custom Errors](#)

V

V8 JavaScript engine

benefits of event-driven model, [Introducing Node](#)

list of V8 options, [Options and Arguments](#)

starting V8 CPU profiler, [Node CLI](#)

versions

Current versus Active LTS status, [Executing Node Code](#)

determining, [Executing Node Code](#)

running multiple, [Executing Node Code](#)

semantic versioning, [Semantic Versioning](#)

vertical scaling, [Strategies of Scalability](#)

Visual Studio Code (VS Code), [Other Tools](#)

W

watch mode, [Options and Arguments](#)

web servers

creating simple, [Using Built-In Modules-Using Built-In Modules](#)

development frameworks, [Frameworks-Frameworks](#)

increasing availability, [Increasing Availability-Increasing Availability](#)

starting in watch mode, [Options and Arguments](#)

Webpack (asset bundling tool), [Node Packages](#), [Module Bundlers](#)

WebStorm (code editor), [Other Tools](#)

weighted round-robin algorithm, [The Cluster Module](#)

window object, [Executing Node Code](#)

Windows, [Executing Node Code](#)

with keyword, [Loading Modules](#)

words, counting, [Options and Arguments](#)

worker mode, [Primary and Worker Processes](#)

worker processes, [The Cluster Module-Primary and Worker Processes](#)

writable streams, [Introducing Streams](#), [Fundamentals of Streams](#)

write method, [Writable Streams](#)

Z

zero-downtime restarts, [Zero-Downtime Restarts-Zero-Downtime Restarts](#)

zlib stream, [Built-In Transform Streams](#)